

Floyd's Cycle Detection Algorithm

Tortoise and Hare — detecting and locating loops in a linked list, in Java

1. The Problem

A singly linked list is normally a chain that ends in null. If a bug (or an intentional circular structure) makes some node's next reference point backward into the list instead of forward to null, the list develops a loop. Walking such a list with an ordinary **while (node != null)** loop never terminates. Before doing anything else with a list of unknown origin, you often need to answer two questions: does a cycle exist, and if so, where does it begin?

The naive fix is to remember every node you've visited (a HashSet of node references) and stop as soon as you see a repeat. That works, but it costs $O(n)$ extra memory. Floyd's algorithm answers the same question using two pointers and no extra data structure at all.

2. Core Idea — Two Runners at Different Speeds

Place two pointers, conventionally called **slow** and **fast**, at the head of the list. On every iteration, slow advances one node while fast advances two nodes. Think of it as two runners on a track: if the track is a straight line, the faster runner simply finishes first and falls off the end (fast becomes null — no cycle). But if the track loops back on itself, the faster runner will eventually come around from behind and pass the slower one, meaning at some point they occupy the same node at the same time.

That single observation is the whole algorithm: if fast ever equals slow again after they've left the starting node, a cycle exists; if fast (or fast.next) hits null first, it doesn't.

3. Why the Pointers Are Guaranteed to Meet

It isn't obvious at first that fast can't simply "hop over" slow inside the cycle without ever landing on the same node. Here's the reasoning:

Once slow has entered the cycle, fast is already somewhere inside it too (fast always reaches the cycle at or before slow, since it moves faster). From that moment on, think about the gap between them, measured as how many steps slow would need to take to reach fast's position going forward around the cycle. Every iteration, slow moves forward by 1 and fast moves forward by 2, so the gap shrinks by exactly 1 each time. A shrinking integer gap measured on a fixed-size loop cannot skip past zero — it must hit zero on some iteration. When the gap is zero, the two pointers are on the same node. So a meeting is not just likely, it is mathematically forced, and it happens within at most one full lap of the cycle after slow enters it.

This is why the algorithm needs no fixed iteration limit and no visited-set: the geometry of moving 1 step versus 2 steps around a loop does all the work.

4. Detecting a Cycle — Java

Using a generic node shape like the one in your own linked-list package (`dsa.linked_list`), detection looks like this:

```
class Node<E> {
    E data;
    Node<E> next;
    Node(E data) { this.data = data; }
}

public class FloydCycle {

    public static <E> boolean hasCycle(Node<E> head) {
        if (head == null) return false;

        Node<E> slow = head;
        Node<E> fast = head;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;

            if (slow == fast) {
                return true;    // pointers met -> cycle exists
            }
        }
        return false;          // fast fell off the end -> no cycle
    }
}
```

Two details matter here. First, the loop condition checks both `fast != null` and `fast.next != null` *before* dereferencing — skipping either check risks a `NullPointerException` on a genuinely acyclic list. Second, the comparison `slow == fast` compares references, not data, which is correct: two different nodes could coincidentally hold equal data, but only the true meeting point is the same object in memory.

5. Finding Where the Cycle Begins

Knowing a cycle exists is often not enough — you usually want the node where it starts, for example to unlink it and restore a proper null-terminated list. This needs a short second phase and a small piece of arithmetic.

Let the distance from the head to the start of the cycle be **a**, the distance from the cycle's start to the meeting point be **b**, and the remaining distance from the meeting point back around to the cycle's start be **c** (so the cycle length is $b + c$). By the time `slow` and `fast` meet, it can be shown from the step counts that **a equals c** (plus whole trips around the cycle, which don't change the destination). That equality is the trick: if you leave one pointer at the meeting point and send a new pointer from the head, moving both one step at a time, they are converging on the same node from two different starting distances that happen to be equal modulo the cycle length — so they land on the cycle's start node at the same time.

```

public static <E> Node<E> findCycleStart(Node<E> head) {
    if (head == null) return null;

    Node<E> slow = head;
    Node<E> fast = head;
    boolean cycleFound = false;

    // Phase 1: detect meeting point inside the cycle
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) {
            cycleFound = true;
            break;
        }
    }

    if (!cycleFound) return null;

    // Phase 2: one pointer from head, one from the meeting point,
    // both stepping by 1 -- they meet at the cycle's start node
    slow = head;
    while (slow != fast) {
        slow = slow.next;
        fast = fast.next;
    }
    return slow;
}

```

6. Complexity

Metric	Value	Why
Time	$O(n)$	Fast pointer traverses at most ~2 laps of the list/cycle combined
Space	$O(1)$	Only two extra references (slow, fast) are ever allocated

The $O(1)$ space bound is the entire reason this algorithm is preferred over a HashSet-based visited-node check when memory matters — the same trade-off you've already run into with Trie versus HashSet approaches in prefix-based problems.

7. Common Pitfalls

- Checking only `fast != null` and forgetting `fast.next != null` before calling `fast.next.next` — throws `NullPointerException` on any list without a cycle.
- Comparing node *data* instead of node *references* — two nodes can legitimately hold equal values without being the same node.
- Starting `slow` and `fast` at different nodes (e.g. `slow` at `head`, `fast` at `head.next`) — this still works but shifts the arithmetic in the cycle-start phase, so it's safer to start both at `head` unless you deliberately account for the offset.

- Forgetting that phase 2 (finding the start) must reset one pointer to **head**, not to the meeting point again — both pointers should move one step at a time from that moment on.

8. Trying It on Your Own LinkedList

Since you already have a hand-built generic `LinkedList<E>` (package `dsa.linked_list.making_my_own`), a good way to internalize this is to add a private helper that deliberately links the tail back to some earlier node, then run `hasCycle` and `findCycleStart` against it before reading any more theory. Because your list already has a documented exception hierarchy (`ListEmpty`, `ListOutOfBounds`), it's also worth deciding what a cycle-aware version of your existing traversal/display methods should do when they encounter one — that's usually the real motivation for having this algorithm on hand.

Related problems worth practicing next: Linked List Cycle II, Happy Number (the same two-pointer idea applied to a sequence of digit-square sums instead of node links), and Find the Duplicate Number (the array version of this exact technique).